Topic: Terraform Variables — Input Variables, Types, `.tfvars`, Precedence, `count`, Maps, Outputs & Locals

(Your notes, cleaned up from dictation + screenshots — Lecture 6)

Quick recall

> **Definition (lecture):** Variables in Terraform are a great way to define **centrally controlled reusable values**. The information in Terraform variables is saved **independently from the deployment plans**, which makes the values easy to read and edit from a single file.
>
> **Definition (reference):** Terraform variables serve as **parameters for your Infrastructure as Code (IaC) configurations**. They act like function arguments that make your code dynamic, flexible and reusable. Instead of hardcoding properties like environment names, IP ranges or machine sizes, you use variables to **pass custom values at runtime**.

There are **three types of variables** in the Terraform world:

Type	Direction	One-liner
Input variables (`variable`)	**Sends** values **to** Terraform	Injected from outside the module to customise behaviour. This is the main meaning of "variables" and **the one we use most of the time**
Output values (`output`)	**Retrieves** values **from** Terraform	Return values that expose internal resource details (instance ID, IP…) to the CLI or to other modules
Local values (`locals`)	Internal only	A local value assigns a **name to an expression** so you can use the name many times within a module instead of repeating the expression. Like private, temporary variables inside the code

---

1. Declaring an input variable

Variables are normally kept in a dedicated `variable.tf` / `variables.tf` file and use the `variable` **block**, which supports several configuration attributes:

```
variable "your_variable_name" {          ──▶ variable name  (used as var.your_variable_name)
  description = "define instance type"   ──▶ description    (human-readable help text)
  type        = string                   ──▶ type           (string, list, map, number …)
  default     = "t2.micro"               ──▶ default value  (used if nothing else sets it)
}
```

Input variables are usually defined by stating a **name, type and default value**. The type and default are not strictly required, but **type is very, very important**: whether it's a string, list, map or number. Declaring it makes Terraform reject wrong values early.

Reference examples:

```
variable "instance_type" {
  type        = string
  description = "The size of the virtual machine"
  default     = "t2.micro"
}

variable "database_password" {
  type        = string
  description = "The master password for the DB cluster"
  sensitive   = true     # Redacts the value from CLI output and logs
}
```

We use input variables to **pass values from outside the configuration or module**. Values can be kept in a file or given as command-line options.

2. Variable types: simple and complex

Input variables support multiple data types, broadly split into **simple (primitive)** and **complex** types.

```
TERRAFORM TYPES
 ├─ string  ┐
 ├─ number  ├─ Primitive (simple) types
 ├─ bool    ┘
 ├─ list / tuple ┐  Complex types (also called structural
 ├─ map / object ┘  or collection types)
 └─ null  ──▶ special value that has no type
```

Primitive types

A primitive type is a simple type that isn't made from any other type.

1. `string`: a sequence of characters representing text, e.g. `"hello"`.

2. `number` : a numeric value, e.g. `10` (decimals like `1.5` are allowed too).
3. `bool` : either `true` or `false` .

Complex types

A complex type **groups multiple values into a single value**. The values can be of the same type or of different types.

List: works like a numbered catalogue of values. Each value is read by its **index**, which starts at 0.

```
variable "users" {
  type    = list(string)
  default = ["root", "user1", "user2"]
}
# usage
username = var.users[0]        # "root"
```

Map: a collection of **keys and values**. It's useful for picking a value from a predefined parameter, e.g. a server size by monthly price:

```
variable "plans" {
  type = map(string)
  default = {
    "5USD"  = "1xCPU-1GB"
    "10USD" = "1xCPU-2GB"
    "20USD" = "2xCPU-4GB"
  }
}
# usage
plan = var.plans["5USD"]       # "1xCPU-1GB"
```

(Also worth knowing: `tuple` is a list with a fixed type per position, `tuple([string, number])` . `object` is a map with a fixed type per key, `object({ name = string, size = number })` . `set` is like a list with no order and no duplicates.)

---

3. Assigning values to input variables and precedence

There are multiple ways to give a variable a value:

Method	Example
Command-line flag `-var`	`terraform apply -var="location=eastus"`
Variable definition file `-var-file`	`terraform plan -var-file="dev-team.tfvars"`
Auto-loaded `.tfvars` **files**	`terraform.tfvars`, `terraform.tfvars.json`, `*.auto.tfvars`, `*.auto.tfvars.json` are loaded automatically if present
Environment variable `TF_VAR_<name>`	`export TF_VAR_location=eastus`. Terraform scans its environment for every variable starting with `TF_VAR_`
`default` in the `variable` block	used when nothing above sets a value
Interactive prompt	if there's no value and no default, Terraform asks: `var.cpu Enter a value:`

A `.tfvars` file is just `name = value` lines:

```
resourceGroupName = "terraformdemo"
location          = "eastus"
```

Variable preferences: the load order

Terraform loads variables in this order. **Each later source overrides the earlier ones**, from **lowest** to **highest**:

```
LOWEST   (default value in the variable block)
  │      1. Environment variables          TF_VAR_cpu
  │      2. terraform.tfvars
  │      3. terraform.tfvars.json
  │      4. *.auto.tfvars / *.auto.tfvars.json   (in alphabetical order)
  ▼      5. Command-line: -var and -var-file     (in the order written; the last one wins)
HIGHEST
```

> Interview line: **"CLI flags beat auto-loaded tfvars, which beat `TF_VAR_` environment variables, which beat the default."** Terraform only prompts when **none** of them gives a value.

---

4. Practical: three ways to organise the code

First case: resource code + variables + values, all in one `main.tf`

```
variable "cpu" {
  default = "t2.micro"
}

resource "aws_instance" "ram" {
  ami           = "ami-066c4849e6b3a1e3d"
  instance_type = "t2.nano"          # step 1: still hard-coded, so the variable is declared
  key_name      = "my_tf_key"
}
```

Step 2: refer to the variable with `var.<name>`:

```
  instance_type = var.cpu            # modern syntax, recommended
# instance_type = "${var.cpu}"       # older interpolation syntax, also works
```

`"${...}"` is only needed when mixing a variable into a **bigger string**, e.g. `"web-${var.env}"`. For a bare reference, use `var.cpu`. `terraform fmt main.tf` tidies the alignment.

Second case: resource code in `main.tf`, variables with values in `variable.tf`

```
# variable.tf
variable "cpu" { default = "t2.small" }
variable "ami" { default = "ami-066c4849e6b3a1e3d" }
variable "key" { default = "my_tf_key" }
```

```
# main.tf
resource "aws_instance" "ram" {
  ami           = var.ami
  instance_type = var.cpu
  key_name      = var.key
}
```

```
-rw-r--r-- 1 root root  98 Sep 28 17:33 main.tf
-rw-r--r-- 1 root root 157 Sep 28 17:30 variable.tf
```

Terraform reads **every `.tf` file in the folder** together, so the file names are only for organisation.

> ⚠️ Typos seen in the lab's first draft: `varaible "cpu"` and `kay_name`. Terraform would reject both: "Unsupported block type" and "Unsupported argument". `terraform validate` catches them.

Third case: resource code, variable declarations and variable values in three separate files

```
# main.tf
resource "aws_instance" "ram" {
  ami           = var.ami
  instance_type = var.cpu
  key_name      = var.key
}

# variable.tf   (declarations only, no values)
variable "cpu" {}
variable "ami" {}
variable "key" {}

# terraform.tfvars   (values only, auto-loaded)
ami = "ami-066c4849e6b3a1e3d"
cpu = "t2.nano"
key = "my_tf_key"
```

Why the third case matters: I can share the **resource code** (`main.tf`) in a public GitHub repository. I can share the **variable file** (`variable.tf`) in a public repository. But I **cannot share the variable values** in any public repository like GitHub. The values (account details, key names, passwords) stay in the `.tfvars` file, which is kept out of Git.

One code base, many teams: `-var-file`

We have one `main.tf` and one `variable.tf`, and we create **two separate tfvars files**, one per team, each with the attributes that team needs for its environment:

```
# prod-team.tfvars
cpu = "t2.larg"                       # typo in the lab; the real type is "t2.large"
ami = "ami-0cca134ec43cf708feefefwer"
key = "adam_keys"

# dev-team.tfvars
cpu = "t2.micro"
ami = "ami-0cca134ec43cf708f"
key = "ram_aws"
```

To tell Terraform which file to use when running `plan` / `apply`, use the command-line argument `-var-file`, which has the highest precedence:

```
terraform plan  -var-file="dev-team.tfvars"
terraform apply -var-file="prod-team.tfvars"
```

*(Custom-named files like these are **not** auto-loaded. Only `terraform.tfvars` and `*.auto.tfvars` are. That's why `-var-file` is needed here.)*

Override one value at runtime: `-var`

Everything is defined (`main.tf`, `variable.tf`, `terraform.tfvars` with `cpu = "t2.small"`), but at runtime a user wants a different value:

```
terraform plan -var="cpu=t2.medium"
```

`t2.medium` **takes precedence** over what's in `terraform.tfvars`.

No value anywhere → Terraform prompts

`terraform.tfvars` has `ami` and `key` but **not** `cpu`. `variable "cpu" {}` is declared with no default. Terraform then **prompts for the value**:

```
[root@Devops-server project1]# terraform plan
var.cpu
  Enter a value:
```

That's handy for learning. In CI there's no one to type, so use `-input=false` to make it fail instead of hanging.

Environment variable: `export TF_VAR_<name>`

```
export TF_VAR_cpu=t2.small
terraform plan                    # no prompt now; cpu = t2.small
```

The catch: `export` only lives in **that shell session**. Open another session, run `terraform plan`, and it prompts again. To make it permanent for your user, put the export in `~/.bashrc`. Remember that this is the **lowest**-priority source, so any tfvars file or `-var` overrides it.

---

5. The production way to define a variable

The empty `variable "cpu" {}` blocks were just for learning. The right way always has **description, type and default** (where a sensible default exists):

```
# define string variable example.....
variable "cpu" {
  description = "cpu type define..."
  type        = string
  default     = "t2.micro"
}
```

6. Number variable + `count`: creating two instances

```
# variable.tf
# define number variable example.....
variable "instance_count" {
  description = "create 2 instances with a number variable"
  type        = number
  default     = 2            # the lab wrote "2"; Terraform converts it to a number, but wri
}
```

```
# main.tf — FIRST ATTEMPT (wrong)
resource "aws_instance" "ram" {
  ami           = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  key_name      = "ram_aws"
  count         = var.instance_count
  tags = {
    Name = "web_server"      # ❌ both instances get the SAME name
  }
}
```

This works, but **both instances get the same Name tag**, which is wrong. We shouldn't follow it. Use the **indexing pattern** with `count.index`:

```
  tags = {
    Name = "web_server_${count.index}"       # web_server_0, web_server_1
  }
```

After `terraform plan` and `terraform apply`, the AWS console showed **2 running t2.micro instances:** `web_server_1` **and** `web_server_2`.

> 📝 `count.index` **starts at 0**, so `"web_server_${count.index}"` gives `web_server_0` and `web_server_1`. Names 1 and 2 (as in the console) come from `"web_server_${count.index + 1}"`. The resources are addressed as `aws_instance.ram[0]` and `aws_instance.ram[1]`.

7. Map variable: pick the AMI by region

AMI IDs are different in every region, so keep them in a map and look them up with another variable:

```
# varibale.tf
variable "amis" {
  type = map
  default = {
    "us-east-1" = "ami-b374d5a5"
    "us-west-2" = "ami-4b32be2b"
  }
}

variable "region" {
  description = "select us-east-1 or us-west-2"
}
```

```
# main.tf
resource "aws_instance" "example" {
  ami           = var.amis[var.region]   # dynamic way to get the right AMI ID for the selec
  instance_type = "t2.micro"
}
```

`region` has no default, so `terraform plan` prompts and shows the **description as the help text**:

```
var.region
  select us-east-1 or us-west-2
  Enter a value: us-east-1
```

(Production touch: add `type = string` and a `validation` block with `contains(["us-east-1","us-west-2"], var.region)` so a typo fails with a clear message instead of "key not found".)

8. Output values: get information back from Terraform

```
resource "aws_instance" "linux" {
  ami           = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  key_name      = "ram_aws"
  tags = {
    Name   = "Test_server"
    source = "terraform"
  }
}
# check some output values..from deployment..

output "instance_id" {
  description = "ID of the EC2 instance"
  value       = aws_instance.linux.id
}

output "instance_ip" {
  description = "Private IP address of the EC2 instance"
  value       = aws_instance.linux.private_ip
}
```

After `apply`, Terraform prints an `Outputs:` section. You can read the values again later without applying:

```
terraform output                    # all outputs
terraform output instance_ip        # one output
terraform output -raw instance_ip   # plain value, handy in scripts
terraform output -json              # machine-readable
```

Outputs are also how a **child module passes values to its parent**, e.g. `module.vpc.vpc_id`. Mark secrets with `sensitive = true`.

9. Local values: names for expressions inside the code

```
locals {
  environment   = "dev"
  instance_type = "t2.nano"
}

resource "aws_instance" "example" {
  ami           = "ami-0cca134ec43cf708f"
  key_name      = "ram_aws"
  instance_type = local.instance_type          # note: local. (singular), not locals.

  tags = {
    Name = "server-${local.environment}"       # server-dev
  }
}
```

Variable vs local:

- A **variable** can be set from **outside** (tfvars, `-var`, env).
- A **local** is fixed **inside** the code and can't be overridden from the command line.

Use locals to avoid repeating an expression, e.g. `local.common_tags` or `"${var.project}-${var.env}"`.

Command reference (this chapter)

```
terraform fmt main.tf                              # tidy formatting
terraform validate                                 # catches typos like varaible / kay_name
terraform plan                                     # prompts for any variable with no value
terraform plan  -var="cpu=t2.medium"               # override one variable (highest precede
terraform plan  -var-file="dev-team.tfvars"        # pick a team's / environment's values
terraform apply -var-file="prod-team.tfvars"
terraform plan  -input=false                       # CI: fail instead of prompting
export TF_VAR_cpu=t2.small                         # env var, this shell only (lowest prece
echo 'export TF_VAR_cpu=t2.small' >> ~/.bashrc     # make it permanent for this user
terraform output                                   # show output values
terraform output -raw instance_ip
terraform console                                  # try expressions: var.amis["us-east-1"]
```

Still to come

- Reading map values with `lookup()`, plus `for_each` with maps (a stable alternative to `count`)

- Variable `validation` blocks and `sensitive` / `nullable`
- Passing variables and outputs between **modules**